SQL
Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**97**
Exact duplicates removed (same question, different wording of punctuation or case)	**0**
Near-duplicates merged (same question asked in different words)	**22**
Unique questions in this PDF	**75**

Section 1: Critical (Must Prepare)	16 questions
Section 2: High Priority	33 questions
Section 3: Medium Priority	20 questions
Section 4: Low Priority / Quick Revision	6 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 16 questions	Asked in almost every SQL round. Be able to write each query by hand, in more than one way where possible, and explain the concept in one or two sentences.
High Priority 33 questions	Frequent follow-ups and practical queries. Practise them after Section 1 using a sample Employee/Orders schema.
Medium Priority 20 questions	Occasionally asked, mostly as a harder follow-up or in SDET-leaning interviews.
Low Priority / Quick Revision 6 questions	Advanced analytical queries that are rarely asked for QA roles. Learn the window-function pattern, not every query.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
Database Concepts & Keys	Clear one-line answers on ACID, keys, normalization and DELETE vs TRUNCATE vs DROP.
Joins & Set Operations	Can you pick the right join, find unmatched rows and explain UNION vs UNION ALL?
Subqueries, Top-N & Ranking	The classic 'second highest salary' family: subquery, LIMIT/TOP, DENSE_RANK, correlated subquery.
Aggregation & GROUP BY	GROUP BY, HAVING vs WHERE, and counts or totals per group, typically written live.
Duplicates & Data Comparison	Finding and deleting duplicates and comparing two tables: very common for QA data validation.
SQL in Testing	How you use SQL as a tester: DB validation, migration, transactions, stored procedures, security.
DML, NULLs, Functions	Everyday query skills: NULL handling, CASE, dates, string functions.
Business Scenario Queries	Turning a business question (latest order, best-selling product) into a correct query.
Advanced Analytics	Window functions and percentile logic; rarely asked for testers, more so for SDET/data roles.

Note: one entry ('SQL Challenge - Queried total balance of employees located in Mumbai') was a resume line; it is now a query task. The 'SQL Query -' prefix was dropped. Repeated questions such as second highest salary, duplicate records and DELETE/TRUNCATE/DROP are merged.

Table of Contents

Section 1: Critical (Must Prepare) (16 questions) . . . 5
Database Concepts & Keys (5) . . . 5
Joins & Set Operations (2) . . . 6
Subqueries, Top-N & Ranking (2) . . . 6
Aggregation & GROUP BY Queries (4) . . . 7
Duplicates & Data Comparison (3) . . . 7

Section 2: High Priority (33 questions) . . . 8
Database Concepts & Keys (2) . . . 8
Joins & Set Operations (4) . . . 8
Subqueries, Top-N & Ranking (7) . . . 9
Aggregation & GROUP BY Queries (5) . . . 10
Duplicates & Data Comparison (1) . . . 10
SQL in Testing (QA Scenarios) (6) . . . 10
DML, NULLs, Functions & Date/String Handling (6) . . . 11
Business Scenario Queries (Orders, Customers, Revenue) (2) . . . 12

Section 3: Medium Priority (20 questions) . . . 13
Database Concepts & Keys (2) . . . 13
Aggregation & GROUP BY Queries (2) . . . 13
DML, NULLs, Functions & Date/String Handling (10) . . . 13
Advanced Analytics & Window Functions (6) . . . 14

Section 4: Low Priority / Quick Revision (6 questions) . . . 16
Advanced Analytics & Window Functions (6) . . . 16

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
Database Concepts & Keys	5 Qs, p.5	2 Qs, p.8	2 Qs, p.13	-	**9**
Joins & Set Operations	2 Qs, p.6	4 Qs, p.8	-	-	**6**
Subqueries, Top-N & Ranking	2 Qs, p.6	7 Qs, p.9	-	-	**9**
Aggregation & GROUP BY Queries	4 Qs, p.7	5 Qs, p.10	2 Qs, p.13	-	**11**
Duplicates & Data Comparison	3 Qs, p.7	1 Qs, p.10	-	-	**4**
SQL in Testing (QA Scenarios)	-	6 Qs, p.10	-	-	**6**
DML, NULLs, Functions & Date/String Handling	-	6 Qs, p.11	10 Qs, p.13	-	**16**
Business Scenario Queries (Orders, Customers, Revenue)	-	2 Qs, p.12	-	-	**2**
Advanced Analytics & Window Functions	-	-	6 Qs, p.14	6 Qs, p.16	**12**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (16 questions)

Asked in almost every SQL round. Be able to write each query by hand, in more than one way where possible, and explain the concept in one or two sentences.

Database Concepts & Keys (5)

1. What are the ACID properties in a database?

   Answer: Atomicity: all steps succeed or none. Consistency: data moves from one valid state to another (constraints hold). Isolation: concurrent transactions do not interfere. Durability: committed data survives a crash.

   Example: Placing an order inserts the order, reduces stock and records payment; if payment fails everything rolls back.

   ```
   BEGIN;
   INSERT INTO orders (order_id, customer_id, amount) VALUES (1001, 7, 999);
   UPDATE products SET stock = stock - 1 WHERE product_id = 55;
   COMMIT;   -- or ROLLBACK; if any step fails
   ```

2. What is the difference between DELETE, TRUNCATE and DROP? **[asked x2 in your bank]**

   Answer: DELETE removes selected rows (DML, WHERE allowed, logged, triggers fire). TRUNCATE removes all rows quickly (no WHERE, resets identity). DROP removes the table itself with its structure.

   Example: Use DELETE to remove one test order, TRUNCATE to clear a staging table, DROP to remove a temporary table.

   ```
   DELETE FROM orders WHERE order_id = 1001;
   TRUNCATE TABLE staging_orders;
   DROP TABLE temp_orders;
   ```

3. What is a primary key? Can a table have multiple primary keys?

   Answer: A primary key uniquely identifies each row; it is unique and not null. A table has only one primary key, but it can be composite (several columns).

   Example: order_id is the primary key of orders; (order_id, product_id) is a composite key in order_items.

   ```
   CREATE TABLE order_items (
     order_id INT, product_id INT, qty INT,
     PRIMARY KEY (order_id, product_id)
   );
   ```

4. What is a foreign key and why is it important in testing?

   Answer: A foreign key links a column to the primary key of another table and enforces referential integrity. In testing, it helps verify there are no orphan records and that cascade rules work.

   Example: orders.customer_id references customers; check for orders whose customer does not exist.

   ```
   SELECT o.order_id FROM orders o
   LEFT JOIN customers c ON c.customer_id = o.customer_id
   WHERE c.customer_id IS NULL;   -- orphan orders
   ```

5. What is normalization and why is it important?

   Answer: Normalization organises data to remove redundancy and update anomalies: 1NF atomic values, 2NF no partial dependency, 3NF no transitive dependency. It keeps data consistent and tells QA where each fact is stored.

   Example: Customer address is stored once in customers, not repeated on every order.

Joins & Set Operations (2)

6. What are joins and their types? Explain INNER, LEFT, RIGHT, FULL and the difference between inner and outer join. **[asked x3 in your bank]**

> **Answer:** A join combines rows from tables on a key. INNER returns matching rows only; LEFT keeps all left rows; RIGHT keeps all right rows; FULL keeps all rows from both. Outer joins keep unmatched rows, inner joins do not.
>
> **Example:** List all customers with their orders, including customers who never ordered (LEFT JOIN).

```
-- Tables: customers(customer_id, name), orders(order_id, customer_id, amount)
SELECT c.name, o.order_id FROM customers c
INNER JOIN orders o ON o.customer_id = c.customer_id;
SELECT c.name, o.order_id FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;
-- FULL JOIN: not in MySQL; use LEFT JOIN UNION RIGHT JOIN
```

7. How do you find records with no matching rows (employees without orders or department, products never sold, rows in one table but not in another)? **[asked x4 in your bank]**

> **Answer:** Use LEFT JOIN with a NULL check on the right key, NOT EXISTS, or EXCEPT/MINUS for rows in one table but not the other. Avoid NOT IN when the column can contain NULL.
>
> **Example:** Customers who never placed an order, and products never sold.

```
SELECT c.customer_id, c.name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL;

SELECT p.product_id FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items i WHERE i.product_id = p.product_id);

SELECT id FROM old_table EXCEPT SELECT id FROM new_table;   -- MINUS in Oracle
```

Subqueries, Top-N & Ranking (2)

8. Write a query to find the second highest salary (also: Nth highest). **[asked x3 in your bank]**

> **Answer:** Use a subquery with MAX, or DENSE_RANK for the Nth highest (handles ties), or ORDER BY with LIMIT/OFFSET.
>
> **Example:** Second highest order amount or second highest salary.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);

SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 2;               -- change 2 to N

SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

9. Retrieve the top N records (top 5 records, top 3 highest paid employees, salaries within the top 3 in the company). **[asked x3 in your bank]**

> **Answer:** Use ORDER BY with LIMIT (MySQL/PostgreSQL), TOP (SQL Server) or FETCH FIRST (Oracle/ANSI). For salaries within the top 3 values use DENSE_RANK.
>
> **Example:** Top 5 most expensive products; employees in the top 3 salary levels.

```
SELECT * FROM products ORDER BY price DESC LIMIT 5;      -- SQL Server: SELECT TOP 5 ...

SELECT * FROM (
  SELECT e.*, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees e
) t WHERE rnk <= 3;
```

Aggregation & GROUP BY Queries (4)

10. What is the difference between WHERE and HAVING?

> **Answer:** WHERE filters rows before grouping and cannot use aggregate functions. HAVING filters groups after aggregation.
>
> **Example:** Customers with more than 5 orders after 2026: WHERE on the date, HAVING on the count.

```
SELECT customer_id, COUNT(*) AS cnt FROM orders
WHERE order_date >= '2026-01-01'
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

11. Find the total salary and the highest salary in each department. **[asked x2 in your bank]**

> **Answer:** Group by department and use SUM and MAX.
>
> **Example:** Total and highest order amount per region follow the same pattern.

```
SELECT dept_id, SUM(salary) AS total_salary, MAX(salary) AS max_salary
FROM employees GROUP BY dept_id;
```

12. Show the count of orders per customer and the total revenue per product. **[asked x2 in your bank]**

> **Answer:** GROUP BY customer with COUNT for orders; for product revenue join order_items and SUM(qty * price).
>
> **Example:** Orders per customer and revenue per product on the sales dashboard.

```
SELECT customer_id, COUNT(*) AS order_count FROM orders GROUP BY customer_id;

SELECT product_id, SUM(qty * price) AS revenue FROM order_items GROUP BY product_id;
```

13. Count how many customers placed more than 5 orders (GROUP BY with HAVING).

> **Answer:** Group by customer and filter groups with HAVING COUNT(*) > 5; wrap in a count if you need the number of such customers.
>
> **Example:** Find repeat buyers who placed more than 5 orders.

```
SELECT COUNT(*) FROM (
  SELECT customer_id FROM orders GROUP BY customer_id HAVING COUNT(*) > 5
) t;
```

Duplicates & Data Comparison (3)

14. How do you find duplicate records in a table? **[asked x2 in your bank]**

> **Answer:** Group by the column and keep groups with more than one row.
>
> **Example:** Find duplicate customer emails.

```
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;
```

15. How do you check for duplicate rows based on multiple columns?

> **Answer:** Group by all the columns that define a duplicate and use HAVING COUNT(*) > 1.
>
> **Example:** Same customer name and phone entered twice.

```
SELECT name, phone, COUNT(*) FROM customers
GROUP BY name, phone HAVING COUNT(*) > 1;
```

16. How do you delete duplicate rows but keep one instance?

> **Answer:** Keep the smallest id per duplicate group and delete the rest. The derived table keeps it valid in MySQL; in SQL Server/PostgreSQL ROW_NUMBER also works. Back up or run in a transaction first.
>
> **Example:** Clean duplicate customers created by a double-submit bug.

```
DELETE FROM customers WHERE customer_id NOT IN (
  SELECT keep_id FROM (
    SELECT MIN(customer_id) AS keep_id FROM customers GROUP BY email
  ) t
);
```

Section 2: High Priority (33 questions)

Frequent follow-ups and practical queries. Practise them after Section 1 using a sample Employee/Orders schema.

Database Concepts & Keys (2)

17. What is an index in SQL?

> **Answer:** An index is a data structure (usually B-tree) that speeds up searches, joins and sorting on a column at the cost of slower writes and extra storage.
>
> **Example:** An index on orders(customer_id) speeds up My Orders queries.

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

18. What is a view in SQL and why might QA use it?

> **Answer:** A view is a saved query that behaves like a virtual table. QA can use views to simplify complex validation queries and to see only the needed columns.
>
> **Example:** A view that joins orders, customers and payments for quick order validation.

```
CREATE VIEW v_order_summary AS
SELECT o.order_id, c.name, o.amount FROM orders o
JOIN customers c ON c.customer_id = o.customer_id;
```

Joins & Set Operations (4)

19. What is a self-join and when is it used?

> **Answer:** A self-join joins a table to itself using aliases, typically for hierarchies such as employee and manager.
>
> **Example:** Show each employee with the manager name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

20. How do you join more than two tables?

> **Answer:** Chain joins one after another on the related keys.
>
> **Example:** Customer name, order and product name in one result.

```
SELECT c.name, o.order_id, p.product_name, i.qty
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN order_items i ON i.order_id = o.order_id
JOIN products p ON p.product_id = i.product_id;
```

21. What is the difference between UNION and UNION ALL?

> **Answer:** UNION combines results and removes duplicates (extra sort/distinct). UNION ALL keeps duplicates and is faster. Both need the same number and compatible types of columns.
>
> **Example:** Combine customer emails from web and app signups: UNION for unique, UNION ALL to count all.

```
SELECT email FROM web_signups
UNION
SELECT email FROM app_signups;
```

22. Find customers who made purchases but never returned products.

> **Answer:** Use EXISTS for customers who have orders and NOT EXISTS against the returns table.
>
> **Example:** Loyal customers who bought and never returned an item.

```
SELECT c.customer_id, c.name FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
  AND NOT EXISTS (SELECT 1 FROM product_returns r WHERE r.customer_id = c.customer_id);
```

Subqueries, Top-N & Ranking (7)

23. How do you select records with salary higher than the department average?

> **Answer:** Compare salary with the department average using a correlated subquery or a join to a grouped average.
>
> **Example:** Orders above the average order value of their region follow the same logic.

```
SELECT * FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

24. How do you get all records except the highest salary?

> **Answer:** Filter salaries lower than the maximum (this excludes every row that has the top salary, including ties).
>
> **Example:** List all orders except the highest-value order.

```
SELECT * FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

25. Write a query to find employees with salary greater than Adam's salary.

> **Answer:** Use a scalar subquery for Adam's salary (use MAX if the name is not unique).
>
> **Example:** Products priced higher than a reference product.

```
SELECT * FROM employees
WHERE salary > (SELECT salary FROM employees WHERE name = 'Adam');
```

26. Find customers with orders above the average order value, or who placed more orders than the average per customer. **[asked x2 in your bank]**

> **Answer:** Compare each order with AVG(amount); for more orders than the average per customer compare each customer's count with the average count.
>
> **Example:** Find high-value orders and customers who order more often than average.

```
SELECT DISTINCT customer_id FROM orders WHERE amount > (SELECT AVG(amount) FROM orders);

SELECT customer_id FROM orders GROUP BY customer_id
HAVING COUNT(*) > (
  SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM orders GROUP BY customer_id) t);
```

27. What is a correlated subquery?

> **Answer:** A correlated subquery refers to a column of the outer query, so it runs once per outer row.
>
> **Example:** Orders above the average amount of the same customer.

```
SELECT * FROM orders o
WHERE amount > (SELECT AVG(amount) FROM orders WHERE customer_id = o.customer_id);
```

28. What is the difference between EXISTS and IN?

> **Answer:** EXISTS checks whether the subquery returns any row and stops at the first match; IN compares a value with a list. NOT IN returns nothing if the list contains NULL, while NOT EXISTS is safe.
>
> **Example:** Customers with orders: EXISTS is usually better for large order tables.

```
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM orders);
```

29. Rank employees by salary within each department (RANK, DENSE_RANK, ROW_NUMBER).

> **Answer:** Use window functions with PARTITION BY department. RANK leaves gaps after ties, DENSE_RANK has no gaps, ROW_NUMBER gives a unique sequence.
>
> **Example:** Rank products by sales within each category.

```
SELECT name, dept_id, salary,
  RANK()       OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rnk,
  ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
FROM employees;
```

Aggregation & GROUP BY Queries (5)

30. Find the number of employees hired each year.

> **Answer:** Group by the year of the hire date (EXTRACT(YEAR FROM ...), or YEAR() in MySQL).
>
> **Example:** New customer signups per year.

```
SELECT EXTRACT(YEAR FROM hire_date) AS yr, COUNT(*) AS hired
FROM employees GROUP BY EXTRACT(YEAR FROM hire_date) ORDER BY yr;
```

31. Get the total revenue and number of orders per region, and monthly sales revenue with order count.

[asked x2 in your bank]

> **Answer:** Group by region for totals; for monthly sales group by a year-month expression (DATE_FORMAT in MySQL, TO_CHAR in PostgreSQL/Oracle).
>
> **Example:** Revenue dashboard by region and month.

```
SELECT region, SUM(amount) AS revenue, COUNT(*) AS orders FROM orders GROUP BY region;

SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(amount) AS revenue, COUNT(*) AS orders
FROM orders GROUP BY DATE_FORMAT(order_date, '%Y-%m') ORDER BY month;
```

32. Calculate the average order value per customer.

> **Answer:** Average the order amount grouped by customer.
>
> **Example:** Average basket value per customer.

```
SELECT customer_id, AVG(amount) AS avg_order_value FROM orders GROUP BY customer_id;
```

33. Identify top-performing departments by average salary.

> **Answer:** Group by department, average the salary and order descending (limit for the top few).
>
> **Example:** Top 3 product categories by average price.

```
SELECT dept_id, AVG(salary) AS avg_salary FROM employees
GROUP BY dept_id ORDER BY avg_salary DESC LIMIT 3;
```

34. SQL challenge: query the total balance of employees located in a city (e.g. Mumbai).

> **Answer:** Filter on the city and SUM the balance column.
>
> **Example:** Total account balance of employees in Mumbai.

```
SELECT SUM(balance) AS total_balance FROM employees WHERE city = 'Mumbai';
```

Duplicates & Data Comparison (1)

35. How do you check whether two tables have the same data?

> **Answer:** Compare in both directions with EXCEPT (MINUS in Oracle); an empty result in both means the data is the same. Also compare row counts.
>
> **Example:** After a data load, confirm staging and final order tables match.

```
(SELECT * FROM table_a EXCEPT SELECT * FROM table_b)
UNION ALL
(SELECT * FROM table_b EXCEPT SELECT * FROM table_a);   -- no rows = same data
```

SQL in Testing (QA Scenarios) (6)

36. What is SQL and why is it important for a QA engineer?

> **Answer:** SQL lets a QA verify backend data directly: check records saved after UI/API actions, set up and clean test data, find duplicates/orphans and validate migrations and reports.
>
> **Example:** After placing an order in the UI, query the orders and payments tables to confirm the data.

37. How do you handle transactions in SQL for testing?

> **Answer:** Use BEGIN/START TRANSACTION, COMMIT, ROLLBACK and SAVEPOINT. For testing, verify that a failure rolls back all changes and use rollback to keep test data clean.
>
> **Example:** Force a payment failure and verify no order or stock change was saved.

```
START TRANSACTION;
UPDATE products SET stock = stock - 1 WHERE product_id = 55;
ROLLBACK;   -- stock must remain unchanged
```

38. What are stored procedures and how can QA test them?

> **Answer:** A stored procedure is precompiled SQL stored in the database and called with parameters. QA tests valid, invalid and boundary inputs, output data, error handling, transaction behaviour and performance.
>
> **Example:** Test apply_coupon(order_id, code) with a valid, expired and empty coupon.

```
CALL apply_coupon(1001, 'SAVE10');
SELECT amount FROM orders WHERE order_id = 1001;   -- verify the discounted amount
```

39. How do you test SQL injection vulnerability?

> **Answer:** Enter payloads such as ' OR '1'='1 in login/search fields (on an authorised test environment) and check the app treats them as plain text with no error or data leak; confirm queries use parameters. Tools: OWASP ZAP, sqlmap.
>
> **Example:** Searching for ' OR '1'='1 must return no results, not all products.

```
-- Test payload in the search box:  ' OR '1'='1' --
-- Safe code uses parameters:
PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE email = ?");
```

40. How do you test data migration when moving from an old DB to a new DB?

> **Answer:** Compare row counts, checksums and sample records between old and new DB, check nulls, duplicates, data types, referential integrity and run EXCEPT queries; also verify performance and rollback plan.
>
> **Example:** After moving customers and orders to the new DB, counts and totals must match.

```
SELECT COUNT(*), SUM(amount) FROM old_db.orders;
SELECT COUNT(*), SUM(amount) FROM new_db.orders;
SELECT order_id FROM old_db.orders EXCEPT SELECT order_id FROM new_db.orders;
```

41. How do you test data encryption in API/DB?

> **Answer:** Check that sensitive columns are stored encrypted or hashed (not plain text), the API uses HTTPS and responses do not expose sensitive data. Passwords should be hashed (e.g. bcrypt), not reversible.
>
> **Example:** The card number column shows ciphertext and the password column shows a hash.

```
SELECT email, password_hash, card_number_enc FROM users WHERE email = 'test@shop.com';
```

DML, NULLs, Functions & Date/String Handling (6)

42. How do you check for NULL values and replace them with a default value (IS NULL, COALESCE/IFNULL)? **[asked x2 in your bank]**

> **Answer:** Test with IS NULL and replace with COALESCE (standard), IFNULL (MySQL) or NVL (Oracle).
>
> **Example:** Show 'N/A' for orders without a coupon code.

```
SELECT order_id, COALESCE(coupon_code, 'N/A') AS coupon FROM orders;
SELECT * FROM orders WHERE coupon_code IS NULL;
```

43. How do you update multiple columns in a single query?

> **Answer:** List the columns in one SET clause separated by commas.
>
> **Example:** Update price and stock of a product together.

```
UPDATE products SET price = 899, stock = 40 WHERE product_id = 55;
```

44. How do you retrieve the last inserted record?

> **Answer:** Order by the key or timestamp descending and take one row; functions such as LAST_INSERT_ID() (MySQL), SCOPE_IDENTITY() (SQL Server) or RETURNING (PostgreSQL) return the id just inserted.
>
> **Example:** Fetch the order your test just created.

```
SELECT * FROM orders ORDER BY order_id DESC LIMIT 1;
SELECT LAST_INSERT_ID();   -- MySQL, in the same session
```

45. How do you fetch records where a column contains a specific substring (LIKE)?

> **Answer:** Use LIKE with wildcards: % for any characters and _ for one character.
>
> **Example:** Search products whose name contains 'phone'.

```
SELECT * FROM products WHERE product_name LIKE '%phone%';
```

46. How do you use a CASE statement in SQL?

> **Answer:** CASE returns different values by condition, like if-else inside a query.
>
> **Example:** Label orders as High, Medium or Low value.

```
SELECT order_id, amount,
  CASE WHEN amount >= 5000 THEN 'High'
       WHEN amount >= 1000 THEN 'Medium'
       ELSE 'Low' END AS order_band
FROM orders;
```

47. How do you get the difference between two dates and find employees hired in the last 30 days? **[asked x2 in your bank]**

> **Answer:** Date functions differ by database. MySQL: DATEDIFF and CURDATE(); PostgreSQL: date subtraction and CURRENT_DATE; SQL Server: DATEDIFF(day, ...).
>
> **Example:** Orders delivered within 3 days; employees hired in the last 30 days.

```
-- MySQL
SELECT name, DATEDIFF(CURDATE(), hire_date) AS days FROM employees;
SELECT * FROM employees WHERE hire_date >= CURDATE() - INTERVAL 30 DAY;
-- PostgreSQL: WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days';
```

Business Scenario Queries (Orders, Customers, Revenue) (2)

48. Get the latest (last) order placed by each customer, along with the order amount; also the first and last order date. **[asked x3 in your bank]**

> **Answer:** Use ROW_NUMBER over each customer ordered by date descending and keep rank 1. For first and last order dates use MIN and MAX with GROUP BY.
>
> **Example:** Show each customer's most recent order amount.

```
SELECT customer_id, order_id, order_date, amount FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn
  FROM orders o
) t WHERE rn = 1;

SELECT customer_id, MIN(order_date) AS first_order, MAX(order_date) AS last_order
FROM orders GROUP BY customer_id;
```

49. Identify the most selling product.

> **Answer:** Sum the quantity sold per product and take the highest (use DENSE_RANK if ties must be included).
>
> **Example:** Best-selling product on the dashboard.

```
SELECT product_id, SUM(qty) AS units FROM order_items
GROUP BY product_id ORDER BY units DESC LIMIT 1;
```

Section 3: Medium Priority (20 questions)

Occasionally asked, mostly as a harder follow-up or in SDET-leaning interviews.

Database Concepts & Keys (2)

50. What is denormalization and when might QA encounter it?

> **Answer:** Denormalization stores redundant data to reduce joins and speed up reads (reports, data warehouses). QA must check that duplicated values stay consistent when the source changes.
>
> **Example:** customer_name copied into the orders table must update when the customer renames.

51. What is the difference between CHAR and VARCHAR?

> **Answer:** CHAR is fixed length (padded with spaces); VARCHAR is variable length. Use CHAR for fixed codes and VARCHAR for names or text; watch for trailing spaces in comparisons.
>
> **Example:** country_code CHAR(2) = 'IN'; product_name VARCHAR(200).

Aggregation & GROUP BY Queries (2)

52. Find the percentage of employees in each department.

> **Answer:** Divide each department's count by the total employee count (or use a window SUM over the group counts).
>
> **Example:** Share of orders per region.

```
SELECT dept_id, COUNT(*) * 100.0 / (SELECT COUNT(*) FROM employees) AS pct
FROM employees GROUP BY dept_id;
```

53. Retrieve the maximum salary difference within each department.

> **Answer:** Take MAX(salary) - MIN(salary) per department.
>
> **Example:** Price spread of products in each category.

```
SELECT dept_id, MAX(salary) - MIN(salary) AS salary_gap FROM employees GROUP BY dept_id;
```

DML, NULLs, Functions & Date/String Handling (10)

54. How do you retrieve distinct department names?

> **Answer:** Use SELECT DISTINCT.
>
> **Example:** List unique product categories.

```
SELECT DISTINCT dept_name FROM departments;
```

55. What is the difference between BETWEEN and IN?

> **Answer:** BETWEEN checks a range (inclusive of both ends); IN checks a list of specific values.
>
> **Example:** Orders from 1000 to 5000 vs orders with status in (PAID, SHIPPED).

```
SELECT * FROM orders WHERE amount BETWEEN 1000 AND 5000;
SELECT * FROM orders WHERE status IN ('PAID', 'SHIPPED');
```

56. How do you retrieve all employees who joined in a given year (2023) or on weekends? **[asked x2 in your bank]**

> **Answer:** For a year use a date range (index friendly) or the year function; for weekends use the weekday function of your database.
>
> **Example:** Orders placed in 2023 and on weekends.

```
SELECT * FROM employees WHERE hire_date >= '2023-01-01' AND hire_date < '2024-01-01';
SELECT * FROM employees WHERE DAYOFWEEK(hire_date) IN (1, 7);   -- MySQL (Sun=1, Sat=7)
-- PostgreSQL: EXTRACT(DOW FROM hire_date) IN (0, 6)
```

57. How do you get the count of rows without using COUNT(*)?

Answer: Use SUM(1) or COUNT(1)/COUNT(column), or read the estimated count from metadata (not exact).

Example: Quick row count of the orders table.

```
SELECT SUM(1) FROM orders;
```

58. How do you check if a table has any rows without fetching the data?

Answer: Use EXISTS or select one row with LIMIT 1, so the DB stops at the first row.

Example: Check the cart table has any rows before running a cleanup.

```
SELECT EXISTS (SELECT 1 FROM orders);      -- MySQL/PostgreSQL
SELECT 1 FROM orders LIMIT 1;
```

59. How do you select random rows from a table?

Answer: Order by a random function and limit: RAND() in MySQL, RANDOM() in PostgreSQL/SQLite, NEWID() in SQL Server.

Example: Pick 5 random orders for manual verification.

```
SELECT * FROM orders ORDER BY RAND() LIMIT 5;      -- MySQL
```

60. How do you convert a string to uppercase and find its length? **[asked x2 in your bank]**

Answer: UPPER() for case and LENGTH() for length (LEN in SQL Server).

Example: Check product names are stored in capital letters and within the length limit.

```
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length FROM customers;
```

61. How do you handle special characters in SQL queries?

Answer: Escape a single quote by doubling it, escape LIKE wildcards with ESCAPE, and always use parameterised queries from code.

Example: Search for a customer named O'Brien or a product with 100% in its name.

```
SELECT * FROM customers WHERE name = 'O''Brien';
SELECT * FROM products WHERE product_name LIKE '%100\%%' ESCAPE '\';
```

62. How do you select only numeric values from a column with mixed data?

Answer: Use a regular expression, or a safe cast that returns NULL for bad values.

Example: Find rows where the quantity field holds only digits.

```
SELECT * FROM orders WHERE qty_text REGEXP '^[0-9]+$';       -- MySQL
-- PostgreSQL: WHERE qty_text ~ '^[0-9]+$'
-- SQL Server: WHERE TRY_CAST(qty_text AS INT) IS NOT NULL
```

63. How do you pivot data in SQL?

Answer: Use conditional aggregation with CASE (works everywhere) or the PIVOT keyword in SQL Server/Oracle.

Example: Show monthly revenue as separate columns per month.

```
SELECT product_id,
  SUM(CASE WHEN month = 1 THEN revenue END) AS jan,
  SUM(CASE WHEN month = 2 THEN revenue END) AS feb
FROM monthly_sales GROUP BY product_id;
```

Advanced Analytics & Window Functions (6)

64. Calculate cumulative revenue by day.

Answer: Aggregate revenue per day, then use SUM() OVER ordered by day.

Example: Running revenue total for the month.

```
SELECT order_date, SUM(daily) OVER (ORDER BY order_date) AS cumulative_revenue
FROM (SELECT order_date, SUM(amount) AS daily FROM orders GROUP BY order_date) t;
```

65. Find the moving average of sales over the last 3 days.

Answer: Use AVG() over a window of the current row and the 2 preceding rows.

Example: 3-day moving average of sales for the trend chart.

```
SELECT order_date,
  AVG(daily) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg
FROM (SELECT order_date, SUM(amount) AS daily FROM orders GROUP BY order_date) t;
```

66. Show product sales distribution (percent of total revenue).

Answer: Divide each product's revenue by the total revenue using SUM() OVER ().

Example: Percent of revenue contributed by each product.

```
SELECT product_id, revenue, revenue * 100.0 / SUM(revenue) OVER () AS pct_of_total
FROM (SELECT product_id, SUM(qty * price) AS revenue FROM order_items GROUP BY product_id) t;
```

67. Find churned customers (no orders in the last 6 months).

Answer: Churned customers have no order in the last 6 months: use the latest order date or NOT EXISTS with a date filter.

Example: Customers to target in a win-back campaign.

```
SELECT c.customer_id, c.name FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.customer_id
    AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH));   -- MySQL syntax
```

68. Calculate revenue generated from new customers (first-time orders).

Answer: Find each customer's first order date and sum the orders placed on that date (or rank 1 by date).

Example: Revenue coming from first-time buyers.

```
SELECT SUM(amount) AS new_customer_revenue FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
  FROM orders o
) t WHERE rn = 1;
```

69. Calculate year-over-year growth in revenue.

Answer: Aggregate revenue by year and compare with the previous year using LAG.

Example: Revenue growth from 2025 to 2026.

```
SELECT yr, revenue,
  (revenue - LAG(revenue) OVER (ORDER BY yr)) * 100.0
    / LAG(revenue) OVER (ORDER BY yr) AS yoy_pct
FROM (SELECT EXTRACT(YEAR FROM order_date) AS yr, SUM(amount) AS revenue
      FROM orders GROUP BY EXTRACT(YEAR FROM order_date)) t;
```

Section 4: Low Priority / Quick Revision (6 questions)

Advanced analytical queries that are rarely asked for QA roles. Learn the window-function pattern, not every query.

Advanced Analytics & Window Functions (6)

70. Retrieve customers who made consecutive purchases (2 days).

> **Answer:** Use LAG to get each customer's previous order date and keep rows where the gap is 1 day.
>
> **Example:** Customers who ordered on two consecutive days.

```
SELECT DISTINCT customer_id FROM (
  SELECT customer_id, order_date,
         LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_date
  FROM orders) t
WHERE DATEDIFF(order_date, prev_date) = 1;   -- MySQL; PostgreSQL: order_date - prev_date = 1
```

71. Find customers who placed orders every month in 2023.

> **Answer:** Filter to 2023, group by customer and keep those with 12 distinct months.
>
> **Example:** Customers who bought every month in 2023.

```
SELECT customer_id FROM orders
WHERE order_date >= '2023-01-01' AND order_date < '2024-01-01'
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM order_date)) = 12;
```

72. Calculate the average time between two purchases for each customer.

> **Answer:** Use LAG to find the gap between consecutive orders per customer and average the gaps.
>
> **Example:** Average repurchase time per customer.

```
SELECT customer_id, AVG(DATEDIFF(order_date, prev_date)) AS avg_days_between   -- MySQL
FROM (SELECT customer_id, order_date,
        LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_date
      FROM orders) t
WHERE prev_date IS NOT NULL GROUP BY customer_id;
```

73. Retrieve the longest gap between orders for each customer.

> **Answer:** Same LAG gap calculation, then take the MAX per customer.
>
> **Example:** Longest inactivity between two orders.

```
SELECT customer_id, MAX(DATEDIFF(order_date, prev_date)) AS longest_gap_days   -- MySQL
FROM (SELECT customer_id, order_date,
        LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_date
      FROM orders) t
WHERE prev_date IS NOT NULL GROUP BY customer_id;
```

74. Find products that contribute to 80% of the revenue (Pareto principle).

> **Answer:** Compute revenue per product, calculate the running share of the total in descending order and keep products until the 80% mark is reached.
>
> **Example:** The few products that bring 80% of sales.

```
SELECT product_id, revenue FROM (
  SELECT product_id, revenue,
         SUM(revenue) OVER (ORDER BY revenue DESC) AS running,
         SUM(revenue) OVER () AS total
  FROM (SELECT product_id, SUM(qty * price) AS revenue FROM order_items GROUP BY product_id) p
) t WHERE running - revenue < 0.8 * total;
```

75. Detect customers above their historical 90th percentile purchase amount, or with revenue below the 10th percentile. **[asked x2 in your bank]**

> **Answer:** Use PERCENT_RANK() (or PERCENTILE_CONT where available). For a customer's own history partition by customer; for low revenue rank customers by total revenue.
>
> **Example:** Flag unusually large orders and low-revenue customers for review.

```
SELECT customer_id, order_id, amount FROM (
  SELECT o.*, PERCENT_RANK() OVER (PARTITION BY customer_id ORDER BY amount) AS pr
  FROM orders o
) t WHERE pr >= 0.9;

SELECT customer_id FROM (
  SELECT customer_id, PERCENT_RANK() OVER (ORDER BY SUM(amount)) AS pr
  FROM orders GROUP BY customer_id) t WHERE pr <= 0.1;
```